

Esercizio 1

Si scriva una classe `NodoBinario` che rappresenta un nodo di un albero binario. Il costruttore prende:

- `val` (numero),
- `left` (opzionale),
- `right` (opzionale).

`left` e `right`, se forniti, devono essere `NodoBinario` oppure `null`, altrimenti lanciare `TypeError`. Se `val` non è un numero, lanciare `TypeError`. Un nodo è foglia se ha entrambi i figli `null`.

Si scriva un generatore `foglieConProfondita(radice)` che:

- prende la radice di un albero (oppure `null`);
- produce una sequenza di coppie [`profondita`, `valore`], dove `valore` è il campo `val` di un nodo foglia e `profondita` la profondità di tale nodo foglia;
- visita l'albero in preordine;

Si assume che radice ha profondità 0.

Se l'albero è vuoto allora `foglieConProfondita` non produce nulla.

Si aggiunga al prototipo della classe `Map` un metodo `incrementa(key, amount)`:

- se la chiave `key` non è presente, inserisce la coppia chiave->valore data da `key`->`amount`;
- se la chiave `key` è presente, incrementa il corrispettivo valore della quantità data da `amount`;
- ritorna la `Map` stessa;

Si aggiunga alla classe `Map` un metodo statico `sommaFogliePerProfondita(radice)` che usando il generatore `foglieConProfondita` restituisca una nuova `Map` tale che:

- le chiavi sono le profondità
- i valori sono la somma dei valori delle foglie a quella profondità
- se radice è `null` allora restituisce una `Map` vuota.

Attenzione: `sommaFogliePerProfondita` non deve modificare l'albero.

Commentare opportunamente il codice.

```
class NodoBinario {
  constructor(val, left = null, right = null) {
    if (typeof val !== "number") {
      throw new TypeError("val deve essere numero");
    }
    if (left !== null && !(left instanceof NodoBinario)) {
      throw new TypeError("left non valido");
    }
    if (right !== null && !(right instanceof NodoBinario)) {
      throw new TypeError("right non valido");
    }

    this.val = val;
    this.left = left;
    this.right = right;
  }
}
```

```

    // Un nodo è foglia se non ha figli
    get isLeaf() {
        return this.left === null && this.right === null;
    }
}

// Generatore per visitare le foglie in preordine
function* foglieConProfondita(nodo, profondita = 0) {
    if (nodo === null) return;

    // Se è foglia, produce la coppia [profondita, valore]
    if (nodo.isLeaf) {
        yield [profondita, nodo.val];
    } else {
        // Visita preordine: sinistra poi destra
        yield* foglieConProfondita(nodo.left, profondita + 1);
        yield* foglieConProfondita(nodo.right, profondita + 1);
    }
}

Map.prototype.incrementa = function(key, amount) {
    if (!this.has(key)) {
        this.set(key, amount);
    }
    else {
        this.set(key, this.get(key) + amount);
    }
    return this;
};

Map.sommaFogliePerProfondita = function(radice) {
    let risultato = new Map();
    if (radice === null) return risultato;

    for (let [profondita, valore] of foglieConProfondita(radice)) {
        risultato.incrementa(profondita, valore);
    }

    return risultato;
};

```

Esercizio 2

Si implementi una classe Ticket che rappresenta un ticket di assistenza tecnica.

Il costruttore prende:

- codice (stringa non vuota),
- priorit  (numero intero tra 1 e 5 inclusi).

Se i parametri non sono validi, lanciare TypeError.

Ogni ticket ha propriet  private:

- codice (sola lettura),
- chiuso (boolean, sola lettura, inizialmente false),
- log (array di stringhe che annotano fatti sul ticket),
- priorit  (lettura/scrittura),

La scrittura di priorit  accetta solo interi tra 1 e 5; altrimenti lancia PrioritaNonValida.

La classe deve fornire:

- aggiungiNota(testo) → aggiunge una nota al log.
- chiudi() → chiude il ticket e registra nel log la stringa "CHIUSURA".

Non   possibile:

- modificare la priorit  se il ticket   chiuso. In tal caso, lanciare ModificaTicketChiuso (con campo aggiuntivo codice);
- aggiungere note se il ticket   chiuso. In tal caso, stessa eccezione.

Si implementi una classe TicketConSLA che, oltre a tutte le propriet  della classe Ticket, ammette anche:

- tempoMassimo (numero > 0, sola lettura) da costruire in fase di istanziamiento;
- tempoTrascorso, inizialmente 0, e successivamente accessibile in lettura e scrittura.

Quando tempoTrascorso viene modificato, se supera tempoMassimo, il ticket viene automaticamente chiuso, e viene lanciata l'eccezione SLASuperato (con campo codice).

Infine, si implementi la funzione incrementaTempo(tickets, delta) che:

- prende tickets (un array eterogeneo), e incrementa tempoTrascorso di delta solo sulle istanze di TicketConSLA;
- restituisce un nuovo array con i ticket che hanno causato SLASuperato, nello stesso ordine dell'array tickets.

Attenzione: non deve interrompersi al primo errore.

Commentare opportunamente il codice.

```
class PrioritaNonValida extends Error {}

class ModificaTicketChiuso extends Error {
  constructor(codice) {
    super("Ticket chiuso");
    this.codice = codice;
  }
}
```

```

}

class SLASuperato extends Error {
  constructor(codice) {
    super("SLA superato");
    this.codice = codice;
  }
}

class Ticket {
  #codice;
  #priorita;
  #chiuso;
  #log;
  constructor(codice, priorita) {
    if (typeof codice !== "string" || codice.length === 0) {
      throw new TypeError("codice non valido");
    }
    if (!Number.isInteger(priorita) || priorita < 1 || priorita > 5) {
      throw new TypeError("priorita non valida");
    }
    this.#codice = codice;
    this.#priorita = priorita;
    this.#chiuso = false;
    this.#log = [];
  }

  get codice() {return this.#codice}
  get priorita() {return this.#priorita}
  get chiuso() {return this.#chiuso}
  get log() { return [...this.#log]}

  set priorita(p) {
    if (this.#chiuso) {
      throw new ModificaTicketChiuso(this.#codice);
    }
    if (!Number.isInteger(p) || p < 1 || p > 5) {
      throw new PrioritaNonValida();
    }
    this.#priorita = p;
    this.#log.push("Cambiata priorità a valore " + p);
  }

  aggiungiNota(testo) {

```

```

    if (this.#chiuso) {
        throw new ModificaTicketChiuso(this.#codice);
    }
    this.#log.push( testo );
}

chiudi() {
    if (!this.#chiuso) {
        this.#chiuso = true;
        this.#log.push("CHIUSURA");
    }
}
}

class TicketConSLA extends Ticket {
    #tempoMassimo;
    #tempoTrascorso;
    constructor(codice, priorit , tempoMassimo) {
        super(codice, priorit );
        if (typeof tempoMassimo !== "number" || tempoMassimo <= 0) {
            throw new TypeError("tempoMassimo non valido");
        }
        this.#tempoMassimo = tempoMassimo;
        this.#tempoTrascorso = 0;
    }

    get tempoMassimo() {return this.#tempoMassimo}
    get tempoTrascorso() {return this.#tempoTrascorso}

    set tempoTrascorso(t) {
        if (typeof t !== "number" || t < 0) {
            throw new TypeError("tempoTrascorso non valido");
        }
        this.#tempoTrascorso = t;
        if (this.#tempoTrascorso > this.#tempoMassimo) {
            super.chiudi();
            throw new SLASuperato(this.codice);
        }
    }
}

function incrementaTempo(tickets, delta) {

```

```

let risultati = [];

if (!Array.isArray(tickets)) return risultati;

for (let t of tickets) {
  if (t instanceof TicketConSLA) {
    try {
      t.tempoTrascorso = t.tempoTrascorso + delta;
    }
    catch (e) {
      if (e instanceof SLASuperato) {
        risultati.push(t);
      }
      //else {
      //  throw e; // errori diversi propagano
      //}
    }
  }
}
return risultati;
}

```

Esercizio 3

Si scriva una classe Parcometro che rappresenta un parcometro personale con credito protetto.

Il costruttore prende due argomenti:

- targa, una stringa non vuota;
- credito, un numero non negativo (default 0).

Se i parametri non sono validi, il costruttore deve lanciare un'eccezione di tipo TypeError.

La classe deve consentire l'accesso in sola lettura al credito corrente disponibile e offrire i seguenti metodi:

- ricarica(euro, motivo) che aggiunge euro al credito;
- paga(euro, motivo) che sottrae euro dal credito;

I metodi ricarica e paga devono lanciare TypeError se euro non è un numero strettamente positivo, oppure motivo non è una stringa.

Il metodo paga deve inoltre lanciare un'eccezione custom CreditoInsufficiente se euro è maggiore del credito disponibile.

I metodi ricarica e paga devono istanziare oggetti della classe Movimento con le seguenti proprietà:

- tipo ("R" = ricarica, "S" = spesa),
- euro (numero ≥ 0),
- motivo (stringa),
- data (istanza Date).

Parcometro fornisce il metodo storico(k) che restituisce un array con gli ultimi k movimenti effettuati (default k = 5), ordinati dal più recente al meno recente.

Attenzione: modificare gli oggetti restituiti da storico non deve alterare le istanze della classe Movimento.

Una operazione è una coppia [p,m] dove p è un riferimento ad una istanza di Parcometro, m è un riferimento ad una istanza di Movimento associato a p. Ogni operazione effettuata dalle istanze della classe Parcometro deve essere registrata in un Set privato che rappresenta il registro globale della classe Parcometro. La classe Parcometro deve inoltre offrire un metodo statico registro() che restituisce un Set di tutte le operazioni [p,m] effettuate su tutte le istanze create.

Attenzione: cancellare elementi dal Set restituito da registro() non deve cancellare elementi dal registro globale della classe Parcometro.

Commentare opportunamente il codice.

```
class CreditoInsufficiente extends Error {
  constructor(message = "Credito insufficiente") {
    super(message);
    this.name = "CreditoInsufficiente";
  }
}

class Movimento{
  constructor(t,e,m,d){
    this.tipo = t // "R" ricarica oppure "S" spesa
    this.euro = e // e>=0
    this.motivo = m // stringa
    this.data = d// date
  }
}

class Parcometro{
  #targa
  #movimenti = []
  #creditoCorrente = 0
  constructor(t,c=0){
    if (typeof t !== "string" || t==="") throw new TypeError("Deve essere stringa non vuota!")
    if (typeof c !== "number" || c<0) throw new TypeError("Deve essere numero non negativo!")
    this.#targa = t
    this.#creditoCorrente = c
  }
  get credito(){ return this.#creditoCorrente}
```

```

    static check(e,m){
        if (typeof m != "string") throw new TypeError("Deve essere
stringa!")
        if ( !Number.isFinite(e) || e<=0) throw new TypeError("Deve essere
numero positivo!")
    }
    static #registri = new Set()
    static registro(){
        return new Set(this.#registri)
    }

    ricarica(euro,motivo){
        Parcometro.check(euro,motivo)
        this.#creditoCorrente += euro
        let mov = new Movimento("R",euro,motivo, new Date())
        this.#movimenti.push(mov)
        Parcometro.#registri.add( [ this, mov ] )
    }
    paga(euro,motivo){
        Parcometro.check(euro,motivo)
        if (euro>this.#creditoCorrente) throw new CreditoInsufficiente
        this.#creditoCorrente -= euro
        let mov = new Movimento("S",euro,motivo, new Date())
        this.#movimenti.push(mov)
        Parcometro.#registri.add( [ this, mov ] )
    }
    storico(k=5){
        return [...this.#movimenti.slice(- k).map((x)=>({...x}))].reverse()
    }
}

```